

Universidad Nacional Autónoma de México

Facultad de Ciencias



Proyecto 1

Construcción de un Analizador Léxico

Compiladores

Equipo: Los Lexers

Integrantes:

Velazquez Vilchis Fernanda Ixchel - 318020941
Elguera Lugo José Luis - 3190071514
Memik Hernández Toprak - 419002354

Semestre: 2026-1

24 de octubre de 2025

Índice

1. Introducción	2
1.1. Qué es un analizador léxico y para qué sirve en un compilador?	2
1.2. Importancia de la teoría de autómatas y lenguajes formales	2
1.3. Objetivo del proyecto	2
2. Fundamentos Teoricos	3
2.1. Definición de lenguaje léxico y token	3
2.2. Transformaciones en la construcción del analizador léxico	3
2.2.1. De Expresiones Regulares (ER) a Autómatas Finito No Determinista con ϵ -transición (AFN ϵ)	3
2.2.2. De AFN ϵ a Autómata Finito Determinista (AFD)	3
2.2.3. De AFD a AFDmin (minimización de estados)	4
2.2.4. De AFDmin a Máquina Discriminadora Determinista (MDD)	4
2.3. Organización del codigo	4
2.4. Repositorio Github	4
3. Desarrollo	5
3.1. Implementación de las funciones en Haskell	5
4. Resultados	14
4.1. Casos de prueba del lenguaje IMP	14
4.2. Ejemplo de entrada (programa IMP)	15
4.3. Pruebas del Analizador Léxico	16
4.4. Salida del lexer	16
5. Conclusiones	17
5.1. Dificultades Encontradas	17
5.2. Lecciones aprendidas	17
6. Referencias	17

1. Introducción

1.1. Qué es un analizador léxico y para qué sirve en un compilador?

El **analizador léxico** es la primera fase de un compilador y tiene como propósito transformar el código fuente escrito por el programador en una secuencia de **componentes léxicos** o **tokens**. Estos tokens son unidades básicas con significado sintáctico, como palabras reservadas, identificadores, números, operadores y delimitadores.

La función principal del analizador léxico es **simplificar el trabajo del analizador sintáctico**, proporcionando una entrada limpia y estructurada. En lugar de procesar directamente caracteres, el analizador sintáctico trabaja con los tokens producidos por el lexer, lo que reduce la complejidad del análisis gramatical y permite detectar errores léxicos en una etapa temprana. “[1]

1.2. Importancia de la teoría de autómatas y lenguajes formales

La **teoría de autómatas y lenguajes formales** constituye el fundamento teórico de los analizadores léxicos y en general, de todo el proceso de compilación. Los **lenguajes formales** permiten describir de manera precisa la estructura de los programas, mientras que los **autómatas finitos** proporcionan un modelo matemático para reconocer los patrones que conforman los tokens de un lenguaje.

Las **expresiones regulares**, que definen los patrones léxicos, son equivalentes a los **autómatas finitos deterministas (AFD)** y **no deterministas (AFN)**. Estas equivalencias garantizan que cada conjunto de tokens pueda representarse mediante un autómata capaz de reconocerlos eficientemente. De esta forma, la implementación de un analizador léxico no solo se basa en la teoría, sino que aplica directamente los resultados formales para convertir expresiones regulares en autómatas y, posteriormente, en programas ejecutables [3].

1.3. Objetivo del proyecto

El objetivo principal de este proyecto es aplicar los conceptos de la teoría de lenguajes formales y autómatas al desarrollo práctico de un analizador léxico para el lenguaje **IMP**. A partir de un conjunto de expresiones regulares que describen los componentes léxicos del lenguaje, se busca construir de manera sistemática las distintas representaciones automáticas —desde el autómata finito no determinista con ϵ -transiciones (AFN ϵ) hasta la Máquina Discriminadora Determinista (MDD)— con el fin de implementar una función **lexer :: Text -> [Token]** que procese el código fuente y produzca la secuencia correspondiente de tokens.

Este proceso permite vincular la teoría con la práctica, mostrando cómo los fundamentos de los lenguajes formales pueden aplicarse directamente en la construcción de herramientas de software reales, en particular dentro del contexto de los compiladores. Además, el proyecto busca desarrollar habilidades de abstracción, modelado formal e implementación funcional en Haskell, así como fomentar el trabajo colaborativo y la documentación técnica rigurosa mediante la elaboración de un reporte con estilo académico.

2. Fundamentos Teoricos

2.1. Definición de lenguaje léxico y token

El **lenguaje léxico** de un lenguaje de programacion es el conjunto de secuencias válidas de símbolos que pueden aparecer en sus programas fuente. Es decir define la estructura elemental del texto que el compilador reconoce antes de analizar su sintaxis y su semántica.

Cada uno de estos elementos léxicos se describe mediante expresiones regulares, que especifican los patrones que componen identificadores, números, operadores, palabras reservadas, delimitadores y otros símbolos. Así, el lenguaje léxico puede entenderse como la unión de los lenguajes regulares que representan cada categoría de componentes léxicos.[2]

Un **token** es la unidad básica de información que el analizador léxico produce al procesar el código fuente.

Token	Descripción Informal	Lexema de muestra
if	caracteres i, f	if
else	caracteres e, l, s, e	else
comparasion	¡o ¿ó ¡= o ¿= ó == ó !=	¡=, !=
numero	cualquier numero constante	3.14159, 0, 6
id	letra seguida por letras y digitos	pi, D2

Cuadro 1: Ejemplos de tokens

2.2. Transformaciones en la construcción del analizador léxico

El proceso de construcción de un analizador léxico a partir de un conjunto de expresiones regulares (ER) se fundamenta en una serie de transformaciones teóricas y automáticas que permiten obtener una representación capaz de reconocer los tokens del lenguaje fuente. Estas transformaciones son las siguientes:

2.2.1. De Expresiones Regulares (ER) a Autómatas Finito No Determinista con ϵ -transición (AFN ϵ)

Las expresiones regulares describen formalmente los patrones que conforman los componentes léxicos del lenguaje. Para reconocerlos de manera automática, cada expresión regular puede transformarse en un autómata finito no determinista con transiciones ϵ .

Si R es una expresión regular, entonces existe un AFN ϵ $N = (Q, \Sigma, \delta, q_0, F)$ tal que:

$$L(N) = L(R)$$

2.2.2. De AFN ϵ a Autómata Finito Determinista (AFD)

El autómata finito determinista (AFD) es una versión del AFN ϵ en la que no existe transiciones en ϵ ni múltiples transiciones posibles para un mismo símbolo.

Esta tranformación garantiza la equivalencia de lenguajes:

$$L(AFD) = L(AFN\epsilon)$$

2.2.3. De AFD a AFDmin (minimización de estados)

Una vez obtenido el AFD, es posible que contenga estados redundantes o equivalentes que reconocen el mismo conjunto de sufijos del lenguaje. El proceso de minimización consiste en identificar y combinar estos estados equivalentes para obtener un **autómata determinista mínimo (AFDmin)**, es decir, el **autómata con el menor número posible de estados que acepta el mismo lenguaje**. Formalmente, el AFDmin $M = (Q', \Sigma, \delta', q'_0, F')$ cumple:

$$L(M) = L(AFD)$$

y no existe otro AFD con menos estados que acepte el mismo lenguaje.

2.2.4. De AFDmin a Máquina Discriminadora Determinista (MDD)

Por ultimo, el **AFD minimizado** se transforma en una **Máquina Discriminadora Determinista (MDD)**, una estructura adaptada al proceso de análisis léxico. La MDD combina en un sol autómata todos los AFDmin correspondientes a los distintos componentes léxicos del lenguaje (identificadores, números, operadores, etc.) para reconocer la cadena más larga posible que forme un token válido.

El resultado final es una función **lexer** que será utilizada por el **analizador sintáctico**.

2.3. Organización del código

```

1
2 Proyecto01/
3 |
4 |-- src/
5 |   |-- ER.hs
6 |   |-- AFNe.hs
7 |   |-- AFN.hs
8 |   |-- AFD.hs
9 |   |-- AFDmin.hs
10 |   |-- MDD.hs
11 |   |-- ER-IMP.hs
12 |   |-- Main.hs
13 |
14 |-- specs/
15 |   |-- IMP.hs
16 |
17 |-- samples/
18 |   |-- imp/      # programas de prueba en IMP
19 |
20 |-- README.md

```

Listing 1: Estructura del proyecto

2.4. Repositorio Github

Click para ir al repositorio del proyecto 1

Ahí estarán las instrucciones para compilar los programas

3. Desarrollo

En este proyecto se implementó un analizador léxico en Haskell para el lenguaje IMP, a partir de un conjunto de expresiones regulares que describen sus componentes léxicos. Se construyó el pipeline completo desde las ER hasta una Máquina Discriminadora Determinista (MDD). Se validó la implementación con programas de ejemplo, confirmando la correcta generación de tokens.

3.1. Implementación de las funciones en Haskell

```

1 module ER where
2 -----
3 -- Definición de tipo de dato para Expresiones Regulares
4 -----
5
6 data ER
7   = Vacio -- Representa el conjunto vaco
8   | Epsilon -- Representa la cadena vaca
9   | Simbolo Char -- Representa un simbolo terminal
10  | Union ER ER -- Representa la unin de dos lenguajes
11  | Concatenacion ER ER -- Representa la concatenacin de dos expresiones
12  | Estrella ER -- Representa el operador de la Estrella de Kleene
13  | Rango Char Char -- Representa un rango de caracteres
14  deriving (Eq)
15
16 instance Show ER where
17   show Vacio = "vacio"
18   show Epsilon = "epsilon"
19   show (Simbolo c) = [c]
20   show (Union l r) = "(" ++ show l ++ " + " ++ show r ++ ")"
21   show (Concatenacion l r) = "(" ++ show l ++ " . " ++ show r ++ ")"
22   show (Estrella e) = show e ++ "*"
23   show (Rango a b) = "[" ++ [a] ++ "-" ++ [b] ++ "]"

```

Listing 2: Expresiones Regulares

```

1 module ER_IMP where
2 import ER
3
4 -----
5 -- Expresiones regulares del lenguaje IMP
6 -----
7
8 -- Enteros: (0 + (1-9)(0-9)* + -(1-9)(0-9)*)
9 erEntero :: ER
10 erEntero =
11   Union
12     (Simbolo '0')
13     (Union
14       (Concatenacion (Rango '1' '9') (Estrella (Rango '0' '9')))
15       (Concatenacion
16         (Simbolo '-')

```

```

17         (Concatenacion (Rango '1' '9') (Estrella (Rango '0' '9'))))
18     )
19 )
20
21 -- Identificadores: ((az + AZ) (az + AZ + 09)*)
22 erIdentificador :: ER
23 erIdentificador =
24     Concatenacion
25         (Union (Rango 'a' 'z') (Rango 'A' 'Z'))
26         (Estrella (Union (Union (Rango 'a' 'z') (Rango 'A' 'Z')) (Rango '0' '9'))))
27
28 -- Operadores aritmticos: (+ + - + * + /)
29 erOpAritmetico :: ER
30 erOpAritmetico =
31     Union (Simbolo '+')
32         (Union (Simbolo '-') (Union (Simbolo '*') (Simbolo '/'))))
33
34 -- Operadores relacionales
35 erOpRelacional :: ER
36 erOpRelacional =
37     Union (Simbolo '=') (Union (Simbolo '>') (Simbolo '<'))
38
39 -- Palabras reservadas
40 erIf, erThen, erElse, erWhile, erDo, erNot, erAnd, erOr, erTrue, erFalse, erSkip :: ER
41 erIf    = stringToER "if"
42 erThen  = stringToER "then"
43 erElse  = stringToER "else"
44 erWhile = stringToER "while"
45 erDo    = stringToER "do"
46 erNot   = stringToER "not"
47 erAnd   = stringToER "and"
48 erOr    = stringToER "or"
49 erTrue  = stringToER "true"
50 erFalse = stringToER "false"
51 erSkip  = stringToER "skip"
52
53 erReservadas :: ER
54 erReservadas =
55     foldr1 Union [erIf, erThen, erElse, erWhile, erDo, erNot, erAnd, erOr, erTrue, erFalse,
56                 erSkip]
57
58 -- Assignacin:
59 erAsignacion :: ER
60 erAsignacion = Concatenacion (Simbolo ':' ) (Simbolo '=')
61
62 -- Delimitadores
63 erDelimitador :: ER
64 erDelimitador = Simbolo ';'
65
66 -- Blancos:
67 erBlanco :: ER
68 erBlanco = Union (Simbolo ' ') (Union (Simbolo '\t') (Union (Simbolo '\n') (Simbolo '\r')
69 ))

```

```

69 -- Eof:
70 erEOF :: ER
71 erEOF = stringToER "eof"
72
73 -- Comentarios:
74 erComentario :: ER
75 erComentario = Concatenacion (Simbolo '-') (Simbolo '-')
76
77 -- =====
78 -- Funcin auxiliar para convertir una cadena a una ER
79 -- =====
80 stringToER :: String -> ER
81 stringToER []      = Epsilon
82 stringToER [c]     = Simbolo c
83 stringToER (c:cs) = Concatenacion (Simbolo c) (stringToER cs)
84
85 -- =====
86 -- Unin de todas las ER
87 -- =====
88 erIMP :: ER
89 erIMP = foldr1 Union
90   [ erEntero
91   , erIdentificador
92   , erOpAritmetico
93   , erOpRelacional
94   , erReservadas
95   , erAsignacion
96   , erDelimitador
97   , erBlanco
98   , erEOF
99   , erComentario
100 ]

```

Listing 3: Expresiones Regulares del lenguaje IMP

```

1 module AFNe where
2 import ER
3
4 -----
5 -- Tipos base
6 -----
7
8 type Estado = String
9 type Simbolo = Char
10
11 -- Transicion: (origen, simbolo (Nothing = ), destinos)
12 type Transicion = (Estado, Maybe Simbolo, [Estado])
13
14 -- Estructura del automata AFN con transiciones epsilon
15 data AFNe = AFNe
16   { estados :: [Estado] -- Conjunto de estados
17   , alfabeto :: [Simbolo] -- Simbolos del alfabeto
18   , transiciones :: [Transicion] -- Lista de transiciones
19   , estadoInicial :: Estado -- Estado inicial

```



```

20  , estadosFinales :: [Estado] -- Lista de estados finales
21  } deriving (Show, Eq)
22
23  -- Funcion principal: ER -> AFNe
24  erAafne :: ER -> AFNe
25  erAafne er = fst (construirAFNe er 0)
26
27  -- Funcion auxiliar: genera nombres de estados
28  nuevoEstado :: Int -> Estado
29  nuevoEstado n = "q" ++ show n
30
31  -- Construccin recursiva del AFNe
32  construirAFNe :: ER -> Int -> (AFNe, Int)
33
34  -- Caso: Conjunto vaco
35  construirAFNe Vacio n =
36    let q0 = nuevoEstado n
37        qf = nuevoEstado (n + 1)
38    in (AFNe [q0, qf] [] [] q0 [qf], n + 2)
39
40  -- Caso: Epsilon
41  construirAFNe Epsilon n =
42    let q0 = nuevoEstado n
43        qf = nuevoEstado (n + 1)
44        trans = [(q0, Nothing, [qf])]
45    in (AFNe [q0, qf] [] trans q0 [qf], n + 2)
46
47  -- Caso: Smbolo individual
48  construirAFNe (Simbolo c) n =
49    let q0 = nuevoEstado n
50        qf = nuevoEstado (n + 1)
51        trans = [(q0, Just c, [qf])]
52    in (AFNe [q0, qf] [c] trans q0 [qf], n + 2)
53
54  -- Caso: Unin (a + b)
55  construirAFNe (Union e1 e2) n =
56    let (a1, n1) = construirAFNe e1 n
57        (a2, n2) = construirAFNe e2 n1
58        q0 = nuevoEstado n2
59        qf = nuevoEstado (n2 + 1)
60        trans =
61          [(q0, Nothing, [estadoInicial a1, estadoInicial a2])] ++
62          transiciones a1 ++ transiciones a2 ++
63          [(f, Nothing, [qf]) | f <- estadosFinales a1 ++ estadosFinales a2]
64        sigma = alfabeto a1 ++ alfabeto a2
65        qs = q0 : qf : (estados a1 ++ estados a2)
66    in (AFNe qs sigma trans q0 [qf], n2 + 2)
67
68  -- Caso: Concatenacin (a . b)
69  construirAFNe (Concatenacion e1 e2) n =
70    let (a1, n1) = construirAFNe e1 n
71        (a2, n2) = construirAFNe e2 n1
72        trans =
73          transiciones a1 ++

```

```

74     transiciones a2 ++
75     [(f, Nothing, [estadoInicial a2]) | f <- estadosFinales a1]
76     sigma = alfabeto a1 ++ alfabeto a2
77     qs = estados a1 ++ estados a2
78     in (AFNe qs sigma trans (estadoInicial a1) (estadosFinales a2), n2)
79
80 -- Caso: Estrella de Kleene (a*)
81 construirAFNe (Estrella e) n =
82     let (a, n1) = construirAFNe e n
83         q0 = nuevoEstado n1
84         qf = nuevoEstado (n1 + 1)
85         trans =
86             transiciones a ++
87             [(q0, Nothing, [estadoInicial a, qf])] ++
88             [(f, Nothing, [estadoInicial a, qf]) | f <- estadosFinales a]
89         qs = q0 : qf : estados a
90     in (AFNe qs (alfabeto a) trans q0 [qf], n1 + 2)
91
92 -- Caso: Rango de caracteres [a-z]
93 construirAFNe (Rango a b) n =
94     let q0 = nuevoEstado n
95         qf = nuevoEstado (n + 1)
96         sigma = [a .. b]
97         trans = [(q0, Just c, [qf]) | c <- sigma]
98     in (AFNe [q0, qf] sigma trans q0 [qf], n + 2)

```

Listing 4: Tipos Base

```

1 module AFN where
2 import AFNe
3 import Data.List (nub)
4
5 data AFN = AFN
6     { estadosAFN :: [Estado]
7     , alfabetoAFN :: [Simbolo]
8     , transicionesAFN :: [(Estado, Simbolo, [Estado])]
9     , estadoInicialAFN :: Estado
10    , estadosFinalesAFN :: [Estado]
11    } deriving (Show, Eq)
12
13
14 -- Funciones auxiliares: movimiento y cierre-
15 mueve :: AFNe -> Estado -> Maybe Simbolo -> [Estado]
16 mueve afn e s =
17     concat [ es | (origen, simb, es) <- transiciones afn
18                 , origen == e
19                 , simb == s ]
20
21 epsilonCierre :: AFNe -> [Estado] -> [Estado]
22 epsilonCierre afn estadosIniciales =
23     let nuevos = nub (estadosIniciales ++ concat [ mueve afn e Nothing | e <-
24     estadosIniciales ])
25     in if length nuevos == length estadosIniciales
26         then nuevos

```

```

26     else epsilonCierre afn nuevos
27
28 -- Eliminacin de transiciones epsilon: AFNe AFN
29 afneToAfn :: AFNe -> AFN
30 afneToAfn afne =
31     let
32         -- Calcular el -cierre de cada estado
33         cierres = [(e, epsilonCierre afne [e]) | e <- estados afne]
34
35         -- Generar nuevas transiciones para cada smbolo del alfabeto
36         nuevasTransiciones =
37             [ (p, a, nub . concat $
38                 [ epsilonCierre afne (mueve afne q (Just a))
39                   | q <- cierreP ])
40             | (p, cierreP) <- cierres
41             , a <- alfabeto afne
42             ]
43
44         -- Filtrar las transiciones sin destino
45         transicionesFiltradas =
46             [ (p,a,es) | (p,a,es) <- nuevasTransiciones, not (null es) ]
47
48         -- Calcular nuevos estados finales
49         nuevosFinales =
50             [ p | (p, cierreP) <- cierres
51               , any ('elem' estadosFinales afne) cierreP
52             ]
53     in
54     AFN
55     { estadosAFN = estados afne
56     , alfabetoAFN = alfabeto afne
57     , transicionesAFN = transicionesFiltradas
58     , estadoInicialAFN = estadoInicial afne
59     , estadosFinalesAFN = nub nuevosFinales
60     }

```

Listing 5: Automatas Finitos no Deterministas

```

1
2 module AFD where
3 import AFN
4 import Data.List (nub, sort)
5 import qualified Data.Set as Set
6
7 type Estado = String
8 type Simbolo = Char
9
10 data AFD = AFD
11     { estadosAFD :: [[Estado]]
12     , alfabetoAFD :: [Simbolo]
13     , transicionesAFD :: [[(Estado, Simbolo, Estado)]
14     , estadoInicialAFD :: Estado
15     , estadosFinalesAFD :: [Estado]
16     } deriving (Show)

```

```

17
18 mueveConjunto :: AFN -> [Estado] -> Simbolo -> [Estado]
19 mueveConjunto afn estados simbolo =
20     nub . concat $
21         [ destino
22         | (origen, s, destino) <- transicionesAFN afn
23         , s == simbolo
24         , origen `elem` estados
25         ]
26
27 afnToAfd :: AFN -> AFD
28 afnToAfd afn =
29     let
30         inicial = sort [estadoInicialAFN afn]
31
32         construir estadosVisitados [] trans = (estadosVisitados, trans)
33         construir estadosVisitados (actual:pendientes) trans =
34             let nuevasTrans =
35                 [ (actual, a, sort (mueveConjunto afn actual a))
36                 | a <- alfabetoAFN afn
37                 ]
38             nuevosEstados =
39                 [ destino
40                 | (_, _, destino) <- nuevasTrans
41                 , not (null destino)
42                 , destino `notElem` (actual : estadosVisitados ++ pendientes)
43                 ]
44             in construir (nub (actual : estadosVisitados))
45                         (pendientes ++ nuevosEstados)
46                         (nub (trans ++ nuevasTrans))
47
48     (estadosDeterministas, transicionesDet) =
49         construir [] [inicial] []
50
51     finales =
52         [ e | e <- estadosDeterministas
53           , any ('elem' estadosFinalesAFN afn) e
54           ]
55     in
56     AFD
57     { estadosAFD = estadosDeterministas
58     , alfabetoAFD = alfabetoAFN afn
59     , transicionesAFD = transicionesDet
60     , estadoInicialAFD = inicial
61     , estadosFinalesAFD = finales
62     }

```

Listing 6: Automatas Finitos Deterministas

```

1
2 module AFDmin where
3
4 import AFD
5 import Data.List (nub, sort)

```

```

6
7 -----
8 -- Definicin del tipo de datos para el AFD minimizado
9 -----
10 data AFDmin = AFDmin
11   { estadosAFDmin :: [[Estado]]
12   , alfabetoAFDmin :: [Simbolo]
13   , transicionesAFDmin :: [[Estado], Simbolo, [Estado]]
14   , estadoInicialAFDmin :: [Estado]
15   , estadosFinalesAFDmin :: [[Estado]]
16   } deriving (Show, Eq)
17
18 -----
19 -- Minimizacion de un AFD
20 -----
21 afdMinimizar :: AFD -> AFDmin
22 afdMinimizar afd =
23   let
24     -- Particin inicial: finales y no finales
25     finales = estadosFinalesAFD afd
26     noFinales = [e | e <- estadosAFD afd, e 'notElem' finales]
27     particionesIniciales = [finales, noFinales]
28
29     -- Refinar particiones hasta que no cambien
30     refinar :: [[Estado]] -> [[Estado]]
31     refinar grupos =
32       let nuevos = concatMap (dividir grupos) grupos
33       in if sort nuevos == sort grupos
34         then grupos
35         else refinar nuevos
36
37     -- Dividir un grupo segn las clases destino
38     dividir :: [[Estado]] -> [Estado] -> [[Estado]]
39     dividir grupos grupo =
40       let clases = nub (map (clase grupos) grupo)
41       in [ [e | e <- grupo, clase grupos e == c] | c <- clases ]
42
43     -- Determinar la clase de un estado segn sus transiciones
44     clase :: [[Estado]] -> [Estado] -> [Int]
45     clase grupos estado =
46       [ case [i | (i, g) <- zip [1..] grupos
47             , any ('elem' g) (buscarDestino estado a)] of
48         [] -> 0      -- Maneja el caso sin destino para evitar head []
49         xs -> head xs
50       | a <- alfabetoAFD afd
51       ]
52
53     -- Buscar los destinos de un conjunto de estados con un smbolo
54     buscarDestino :: [Estado] -> Simbolo -> [[Estado]]
55     buscarDestino es a =
56       [ destino
57       | (origen, simbolo, destino) <- transicionesAFD afd
58       , any ('elem' origen) es
59       , simbolo == a

```

```

60     ]
61
62     -- Aplicar refinamiento
63     particionesFinales = refinar particionesIniciales
64
65     -- Asociar cada estado con su clase final
66     buscarClase :: [Estado] -> [Estado]
67     buscarClase e = head [g | g <- concat particionesFinales, any ('elem' g) e]
68
69     -- Construir transiciones del AFDmin
70     nuevasTransiciones =
71         nub [ (buscarClase origen, simbolo, buscarClase destino)
72             | (origen, simbolo, destino) <- transicionesAFD afd
73             , not (null destino)
74             ]
75
76     -- Determinar estado inicial y finales del AFDmin
77     inicial = buscarClase (estadoInicialAFD afd)
78     nuevosFinales =
79         [ g
80         | g <- concat particionesFinales
81         , any (any ('elem' g)) finales
82         ]
83
84     in
85     AFDmin
86     { estadosAFDmin = concat particionesFinales
87     , alfabetoAFDmin = alfabetoAFD afd
88     , transicionesAFDmin = nuevasTransiciones
89     , estadoInicialAFDmin = inicial
90     , estadosFinalesAFDmin = nuevosFinales
91     }

```

Listing 7: Automatas Finitos Deterministas minimo

```

1 module MDD where
2
3 import AFDmin
4 import Data.List (find)
5 import AFD
6
7 -----
8 -- Definición de la Máquina Discriminadora Determinista (MDD)
9 -----
10 data MDD = MDD
11     { estadosMDD :: [[Estado]]
12     , alfabetoMDD :: [Simbolo]
13     , transicionesMDD :: [[Estado], Simbolo, [Estado]]
14     , inicialMDD :: [Estado]
15     , finalesMDD :: [[Estado]]
16     } deriving (Show, Eq)
17
18 -----
19 -- Conversin: AFDmin MDD

```

```

20 -----
21 afdminToMDD :: AFDmin -> MDD
22 afdminToMDD afdmin = MDD
23   { estadosMDD = estadosAFDmin afdmin
24   , alfabetoMDD = alfabetoAFDmin afdmin
25   , transicionesMDD = transicionesAFDmin afdmin
26   , inicialMDD = estadoInicialAFDmin afdmin
27   , finalesMDD = estadosFinalesAFDmin afdmin
28   }
29 -----
30 -----
31 -- Ejecucin de la MDD con una cadena de entrada
32 -----
33 ejecutarMDD :: MDD -> String -> Bool
34 ejecutarMDD mdd = recorrer (inicialMDD mdd)
35   where
36     recorrer estadoActual [] =
37       estadoActual 'elem' finalesMDD mdd
38
39     recorrer estadoActual (c:cs) =
40       case find (\(origen, simbolo, _) -> origen == estadoActual && simbolo == c)
41         (transicionesMDD mdd) of
42         Just (_, _, destino) -> recorrer destino cs
43         Nothing               -> False

```

Listing 8: Maquina Discriminadora Determinista

4. Resultados

4.1. Casos de prueba del lenguaje IMP

```

1 -----
2 Pruebas de ER:
3 -----
4 a
5 (a + b)
6 (a . b)
7 a*
8 [a-c]
9 -----
10 -----
11 Pruebas de AFNe:
12 -----
13 AFNe {estados = ["q0","q1"], alfabeto = "a", transiciones = [("q0",Just 'a',["q1"])],
14     estadoInicial = "q0", estadosFinales = ["q1"]}
15 AFNe {estados = ["q4","q5","q0","q1","q2","q3"], alfabeto = "ab", transiciones = [("q4",
16     Nothing,["q0","q2"]),("q0",Just 'a',["q1"]),("q2",Just 'b',["q3"]),("q1",Nothing,["q5"
17     ]),("q3",Nothing,["q5"])], estadoInicial = "q4", estadosFinales = ["q5"]}
18 -----
19 Pruebas de AFN:

```

```

18 -----
19 AFN {estadosAFN = ["q0","q1"], alfabetoAFN = "a", transicionesAFN = [("q0",'a',"q1")],
    estadoInicialAFN = "q0", estadosFinalesAFN = ["q1"]}
20 AFN {estadosAFN = ["q4","q5","q0","q1","q2","q3"], alfabetoAFN = "ab", transicionesAFN =
    [("q4",'a',"q1","q5"),("q4",'b',"q3","q5"),("q0",'a',"q1","q5"),("q2",'b',"q3",
    "q5")], estadoInicialAFN = "q4", estadosFinalesAFN = ["q5","q1","q3"]}
21
22 -----
23 Pruebas de AFD:
24 -----
25 AFD {estadosAFD = [{"q1"},{"q0"}], alfabetoAFD = "a", transicionesAFD = [{"q0",'a',"q1"},
    [{"q1",'a',[]}], estadoInicialAFD = [{"q0"}], estadosFinalesAFD = [{"q1"}]}
26 AFD {estadosAFD = [{"q3","q5"},{"q1","q5"},{"q4"}], alfabetoAFD = "ab", transicionesAFD =
    [{"q4",'a',"q1","q5"},{"q4",'b',"q3","q5"},{"q1","q5",'a',[]},{"q1","q5",
    'b',[]},{"q3","q5",'a',[]},{"q3","q5",'b',[]}], estadoInicialAFD = [{"q4"}],
    estadosFinalesAFD = [{"q3","q5"},{"q1","q5"}]}
27
28 -----
29 Pruebas de AFDmin:
30 -----
31 AFDmin {estadosAFDmin = [{"q1"},{"q0"}], alfabetoAFDmin = "a", transicionesAFDmin = [{"q0"},
    'a',"q1"]], estadoInicialAFDmin = [{"q0"}], estadosFinalesAFDmin = [{"q1"}]}
32 AFDmin {estadosAFDmin = [{"q3","q5"},{"q1","q5"},{"q4"}], alfabetoAFDmin = "ab",
    transicionesAFDmin = [{"q4"},'a',"q3","q5"], [{"q4"},'b',"q3","q5"]],
    estadoInicialAFDmin = [{"q4"}], estadosFinalesAFDmin = [{"q3","q5"},{"q1","q5"}]}
33
34 -----
35 Pruebas de MDD:
36 -----
37 MDD {estadosMDD = [{"q1"},{"q0"}], alfabetoMDD = "a", transicionesMDD = [{"q0"},'a',"q1"},
    [{"q1"},'a',[]], inicialMDD = [{"q0"}], finalesMDD = [{"q1"}]}
38 MDD {estadosMDD = [{"q3","q5"},{"q1","q5"},{"q4"}], alfabetoMDD = "ab", transicionesMDD =
    [{"q4"},'a',"q3","q5"], [{"q4"},'b',"q3","q5"]], inicialMDD = [{"q4"}], finalesMDD
    = [{"q3","q5"},{"q1","q5"}]}
39
40 -----
41 Ejecucin de MDD con cadenas de prueba:
42 -----
43 MDD1 acepta 'a'? True
44 MDD1 acepta 'b'? False
45 MDD2 acepta 'a'? True
46 MDD2 acepta 'b'? True
47 MDD2 acepta 'ab'? False
48 MDD2 acepta 'ba'? False

```

Listing 9: Pruebas Expresiones Regulares

4.2. Ejemplo de entrada (programa IMP)

```

1 -----
2 Pruebas de ER_IMP:
3 -----
4

```



```

5 Expresin regular para enteros:
6 (0 + ((([1-9] . [0-9]*) + (- . ([1-9] . [0-9]*))))))
7
8 Expresin regular para identificadores:
9 ((([a-z] + [A-Z]) . ((([a-z] + [A-Z]) + [0-9]))*))
10
11 Expresin regular para operadores aritmticos:
12 (+ + (- + (* + /)))
13
14 Expresin regular para operadores relacionales:
15 (= + (< + (> + (((< . =) + (> . =)))))
16
17 Expresin regular para palabras reservadas:
18 (((i . f) + ((t . (h . (e . n))) + ((e . (l . (s . e))) + ((w . (h . (i . (l . e)))) + ((d
    . o) + ((n . (o . t)) + ((a . (n . d)) + ((o . r) + ((t . (r . (u . e))) + ((f . (a .
    (l . (s . e)))) + (s . (k . (i . p)))))))))
19
20 Expresin regular para asignacin:
21 (: . =)
22
23 Expresin regular para delimitadores:
24 ;
25
26 Expresin regular para comentarios:
27 ((- . -) . ([A-Z] + ([a-z] + [0-9]))*)
28
29 Expresin regular global del lenguaje IMP:
30 ((0 + ((([1-9] . [0-9]*) + (- . ([1-9] . [0-9]*)))) + ((([a-z] + [A-Z]) . ((([a-z] + [A-Z])
    + [0-9]))*) + ((+ + (- + (* + /))) + ((= + (< + (> + (((< . =) + (> . =))))) + (((i . f
    ) + ((t . (h . (e . n))) + ((e . (l . (s . e))) + ((w . (h . (i . (l . e)))) + ((d . o
    ) + ((n . (o . t)) + ((a . (n . d)) + ((o . r) + ((t . (r . (u . e))) + ((f . (a . (l
    . (s . e)))) + (s . (k . (i . p))))))))) + (((: . =) + (; + (( + (
31 ))) + ((((- . -) . ([A-Z] + ([a-z] + [0-9]))*) + (e . (o . f)))))))))

```

Listing 10: Pruebas con expresiones regulares de IMP

4.3. Pruebas del Analizador Léxico

```
1 Codigo fuente:
2 if x := 3 + 5; while y < 10 do skip;
```

Listing 11: Código Fuente

4.4. Salida del lexer

```
1 [TId "if",TId "x",TAssign,TInt 3,TPlus,TInt 5,TSemicolon,TId "while",TId "y",TLt,TInt 10,  
   TId "do",TId "skip",TSemicolon,TEOF]
```

Listing 12: Tokens Generados

5. Conclusiones

5.1. Dificultades Encontradas

El proyecto nos presentó un gran reto, ya que no sabíamos como iniciar. Entonces nos enfocamos en investigar sobre los objetivos solicitados por el profesor.

El determinar las expresiones regulares para describir correctamente los tokens, hacer que las reglas no sean ambiguas para poder diferenciar entre palabras reservadas e identificadores.

El hacer las transformaciones $\mathbf{ER} \rightarrow \mathbf{AFN}_\epsilon \rightarrow \mathbf{AFD} \rightarrow \mathbf{AFDmin} \rightarrow \mathbf{MDD}$ requieren una comprensión para hacer bien la implementación y que no tengan errores.

Otro tema es el tiempo que tarda en cargar el programa, si queremos checar algun error debemos esperar mucho

5.2. Lecciones aprendidas

- La complejidad de transformar conceptos teóricos en una herramienta funcional como los es el analizador léxico
- La investigación a profundidad siempre sera importante para poder disminuir la dificultad de cualquier trabajo.

6. Referencias

- [1] AV Aho, MS Lam, R Sethi, and JD Ullman. Lexical analysis. *Compilers: Principles, Techniques, and Tools. 2nd ed. Pearson Education: Inc*, pages 109–90, 2006.
- [2] Keith D Cooper and Linda Torczon. *Engineering a compiler*. Morgan Kaufmann, 2022.
- [3] John E Hopcroft, Rajeev Motwani, and Jeffrey D Ullman. Introduction to automata theory, languages, and computation. *Acm Sigact News*, 32(1):60–65, 2001.